# Project - n-body: Move U'r Body

Martin Hart 21200027
Soufiane El mouahid 28626439

January 20, 2025

# Contents

**1 Introduction** **2**

**2 Optimizations** **3**

**3 Computer Architecture** **12**

**4 Experimentation Results** **13**

**5 Conclusion** **17**

# 1 Introduction

This project explores the simulation of a galaxy where celestial bodies interact through gravitational forces, following Newton's laws of motion and gravitation. Each body is initialized with unique masses and positions to replicate the dynamic behavior of planetary systems. The simulation represents an n-body problem, a classical computational challenge that requires efficient methods to calculate the interactions among a large number of bodies.

To achieve this, we aim to optimize the computational workload and maximize throughput, leveraging both CPU and GPU resources. By applying principles and techniques learned during this course, the project delves into various optimization strategies to tackle the n-body problem, balancing precision and performance.

The simulator is developed in C++, structured around object-oriented programming principles. Bodies and their computations are represented through classes, with each class encapsulating a distinct computational approach. The project integrates with MUrB (Move U'r Body), an application providing a flexible interface for implementation. This interface allows us to extend the simulator by creating new classes, each showcasing unique methodologies for simulating gravitational interactions.

# 2 Optimizations

In this section, we are exploring each implementation, highlighting the specific optimizations applied and how we think they are supposed to contribute to improving the performance and efficiency of the n-body simulation.

## CPU

### Optim cpu+optim

### Description

In this first implementation, we introduce two groups of optimizations: the reflexive optimization and a set of general, unnamed optimizations. The latter will be common to all subsequent implementations in this project, providing a consistent baseline of improvements. Reflexive optimization, on the other hand, is specific to this implementation and is added selectively in other versions to create a "reflexive variant" of those implementations.

**Unnamed Optimizations (Shared Across All Implementations)**

- **Pre-computation of constants:** We compute `softSquared` and `G.mn` outside the main loop to avoid redundant calculations.
- **Avoiding `std::pow` for small exponents:** For better performance, we replace `std::pow` with simpler arithmetic operations where applicable.

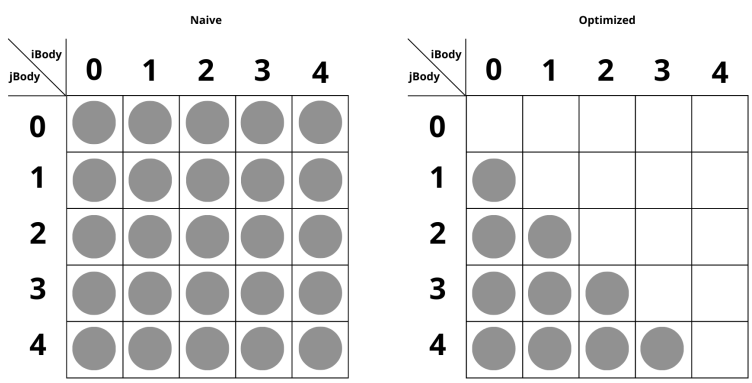
**Reflexive Optimization (Specific to This Implementation)** Each body pair is processed only once. For a given pair (`iBody`, `jBody`), the effects on both bodies are computed simultaneously within the same iteration. This is achieved by:

- **Shared computation:** The acceleration contribution for `jBody` is calculated during the same iteration as `iBody`. Instead of recalculating the position differences ($\vec{r_{ij}}$) or the distance $(\|\vec{r_{ij}}\|^2 + e^2)^{3/2}$, we reuse the previously computed values. Using `iBody`'s mass, we compute a new acceleration $a_j$ and apply the negated distance vector $\vec{r_{ij}}$ scaled by $a_j$. The result is then directly added to `jBody`'s acceleration vector.

- **Efficient pair traversal:** By initializing `jBody` to `iBody + 1`, we ensure that each pair is visited only once. This prevents redundant calculations for the same pair in reverse order (e.g., $(iBody, jBody)$ and $(jBody, iBody)$). The increment avoids self-pairs ($iBody == jBody$), as bodies do not exert gravitational forces on themselves.
- **Loop termination optimization:** Limiting the outer loop to `Nbodies - 1` ensures that the last body does not unnecessarily iterate over an empty range. This avoids redundant boundary checks for non-existent pairs.

### Expected Performance

- **Iteration Reduction:** The total number of iterations decreases significantly, from $n^2$ to $n(n-1)/2$, effectively halving the workload.
- **Increased FLOPs per Iteration:** The floating-point operations (FLOPs) per iteration increase modestly, from 20 to 25, due to the added computation to handle both bodies in each pair simultaneously.
- **Performance Gain:** These optimizations are expected to nearly double the overall performance, as the significant reduction in iterations outweighs the slight increase in per-iteration workload.



**SIMD cpu+simd**

**Description**

**Data Access Management**

- **Extending Temporary Structures:** A temporary structure (`tmp`) is created to handle potential out-of-bound accesses when the number of bodies (`Nbodies`) is not a multiple of the SIMD register size. This structure includes only the necessary members $(q_x, q_y, q_z)$ used in the computations, with values beyond `Nbodies` uninitialized as they do not affect the calculations. The size of this structure is extended to `index`, the next multiple of the SIMD register size, ensuring safe access during computations. For cases where `Nbodies` is a multiple, no extension is performed, and the base data structure is referenced directly.
- **Extending the Precomputed $G \cdot m$ Table:** The precomputed $G \cdot m$ table, common to all implementations, is extended to `index`, with additional entries initialized to zero. This ensures that, since these values are used as the numerator in division operations, calculations for out-of-bound indices result in zero contributions to the results.

**Computation Optimizations**

- **Batch Processing via SIMD:** SIMD registers are used to process multiple `jBodies` simultaneously for a given `iBody`, enabling efficient parallelization and optimization of computations.
- **Efficient Squared Distance Calculation:** The squared distance $r_{ij}^2 = r_{ij,x}^2 + r_{ij,y}^2 + r_{ij,z}^2$ is computed using two `fmadd` instructions. This reduces the number of operations.
- **Efficient Computation of $(\|\vec{r_{ij}}\|^2 + e^2)^{3/2}$:** This is efficiently computed using SIMD registers and the `mipp::sqrt()` function, avoiding the costly `std::pow()` function.

**Reduction Optimization**

- **Buffer Registers for Accelerations:** At the start of each `iBody` iteration, temporary buffers $(a_x, a_y, a_z)$ are initialized to zero within SIMD registers. These buffers accumulate the accelerations for the current

iBody across all jBodies. After the loop over jBodies, a single reduction is performed, and the result is written to the global acceleration table. This approach minimizes global memory writes and register reductions, which are expensive in terms of performance.

### Expected Performance

- **Iteration Reduction:** The inner loop iterations decrease significantly, processing jBodies in chunks of mipp::N<float>(), reducing the workload by a factor of the SIMD width.
- **Increased FLOPs per Iteration:** The floating-point operations (FLOPs) per iteration rises from 18 to 72 for fully utilized SIMD registers but are amortized across multiple jBodies, maintaining efficiency.
- **Performance Gain:** These optimizations leverage SIMD parallelism to significantly improve throughput, with huge expected speedups resulting from reduced iteration counts and efficient vectorized computations.

### Optim cpu+optim+simd

**Description** In the reflexive SIMD optimization, the primary challenge was the varying number of jBody iterations for each iBody, making it significantly harder to handle the SIMD overflow efficiently. Instead of extending nBodies to fit the SIMD registers as in the non-reflexive version, this implementation adopts a two-loop strategy:

- **Dynamic Partitioning with Index:** For each iBody, the computation is divided into two parts. First, as many jBody iterations as possible are processed using SIMD registers, determined dynamically by recalculating the Index value for each iBody. Index specifies how many jBody values can fit within SIMD registers, considering the remaining bodies to process.
- **Sequential Fall-Back:** If there are leftover bodies that cannot fit into SIMD registers, they are processed sequentially. This ensures correctness while maintaining the benefits of SIMD parallelism for the majority of the iterations.

- **Dynamic Index Adjustment:** The `Index` is recalculated dynamically for each `iBody`, starting from `iBody + 1` and indicating the position where the SIMD processing ends. If all `jBody` values for a given `iBody` fit into SIMD registers, `Index` equals `nBodies`. Otherwise, it marks the point where the sequential loop begins.

This adaptive approach maintains high performance while handling the complexities introduced by the reflexive optimization. It balances SIMD efficiency with the flexibility needed to handle variable loop ranges.

### Expected performance

- **Iteration Reduction:** Similar to the base SIMD implementation, the reflexive optimization theoretically reduces the number of iterations from $n(n-1)/2$, effectively halving the workload.
- **Increased Overhead:** The recalculation of `Index` for each `iBody` introduces a theoretical overhead, as it adds extra steps to determine which bodies cannot fit into SIMD registers. However, this overhead is expected to be minimal compared to the gains from reduced iterations and SIMD acceleration.
- **Performance Gain:** This implementation combines reflexive computation with dynamic SIMD parallelism. The expected result is significant theoretical speedups compared to the base SIMD implementation, particularly for larger `nBodies`. However, these speedups are purely anticipated and not based on actual measurements.

### OpenMP Implementations [implems]+omp

**Description** For all three implementations (`Optim OMP`, `SIMD OMP`, and `Optim SIMD OMP`), OpenMP was utilized to multithread the `iBody` loop, distributing the workload across multiple threads for better parallelism. However, since results are stored in `this->accelerations`, which is shared among threads, this caused precision issues. Even when using `#pragma omp critical` to avoid concurrency, the imprecision persisted.

To address this, a local buffer was created for each thread at the beginning of the `iBody` loop. Each thread stored its computed results in this local buffer. At the end of the `jBody` loop, threads wrote their results to

`this->accelerations` within a critical section, ensuring thread safety and accurate results. This approach resolved the concurrency and precision issues.

For the `Optim SIMD` implementation with OpenMP, the `Index` variable had to be made private to each thread, as it is recalculated at the start of each `iBody` loop. This ensured that threads independently computed the necessary values without interfering with one another.

### Expected performance

- **OpenMP Parallelism:** Distributing `iBody` across the Jetson TX2's 6 CPU cores should yield significant speedups, limited by workload balancing and memory bandwidth.
- **Thread-Local Buffers:** Avoiding contention on `this->accelerations` with thread-local buffers improves precision and reduces synchronization overhead.
- **SIMD + OpenMP:** Combining SIMD and multithreading maximizes resource utilization, creating an absolute beast of an implementation expected to deliver exceptional performance.

### POSIX Implementations cpu(+optim)+posix

**Description** The POSIX implementation is based on the naive version, enhanced with the common optimizations. Task distribution is achieved by dividing the workload across multiple processes. Each process computes its assigned workload and stores the results in shared allocated memory. After all processes complete their computations, the parent process retrieves the results from the shared memory.

For the POSIX Optimized version, the same methodology is followed, but it also incorporates the reflexive optimization, further reducing redundant computations for improved efficiency.

### Expected performance

- **Task Parallelism:** By leveraging multiple processes, this implementation should effectively utilize available CPU cores. Each process is ex-

pected to independently compute its portion of the workload, reducing overall computation time compared to the single-threaded approach.

- **Shared Memory Overhead:** While shared memory introduces some overhead for synchronization and data retrieval, this is expected to have minimal impact on performance given the simplicity of the data exchange mechanism.
- **POSIX Optimized:** Incorporating the reflexive optimization should further reduce computational redundancy.

**Real performance** The actual performance of both the POSIX and POSIX Optimized implementations was unexpectedly poor, to the point that we have decided not to include these implementations in the results section of the experimental evaluations. The root cause lies in an oversight during design: the overhead associated with creating and destroying processes for each iteration of the program is extremely costly. In hindsight, this oversight seems obvious, as such operations inherently introduce significant overhead. This completely negates any potential performance benefits, resulting in no measurable improvement compared to the naive implementation without optimizations.

## GPU

### CUDA gpu+cuda

**Description** In the GPU implementation, based on the naive version with the common optimizations, the nested loops are removed, and the total computations are represented as a matrix. Each thread is assigned a single computation from this matrix. To handle cases where the grid dimensions do not align perfectly with the number of bodies (`nBodies`), we ensure the grid size either matches or slightly exceeds `nBodies`. Any threads falling outside the valid range are made idle through a conditional check (`if`) in the kernel.

To accumulate accelerations in the shared buffer (`accelerations`) used by all GPU threads, we utilize CUDA's `atomicAdd` function, ensuring proper updates without concurrency issues. This implementation takes full advantage of CUDA's parallel processing capabilities to distribute the computational workload across the GPU.

**Expected Performance** This GPU implementation should vastly outperform all CPU-based implementations, including those with OpenMP, due to its ability to parallelize $n^2$ computations across 256 CUDA cores. Compared to the naive CPU version without any optimizations, the expected speedup is several orders of magnitude. However, the use of `atomicAdd` may introduce slight bottlenecks, particularly for very large `nBodies`.

### Optim CUDA gpu+optim+cuda

**Description** This implementation builds upon the previous GPU version by integrating the reflexive optimization directly into the kernel. The processing thread condition is modified so that `iBody` ranges from 0 to `nBodies-1`, and `jBody` starts at `iBody+1`. This ensures that each body pair is computed only once, avoiding redundant calculations for symmetric interactions. Threads outside these adjusted ranges are inactive, maintaining efficiency.

**Expected Performance** By combining the GPU's parallelism with the reflexive optimization, this implementation should significantly outperform the basic GPU version, halving the total number of computations while maintaining high throughput.

## Hetero

### SIMD OMP CUDA hetero

**Description** The Heterogeneous Implementation combines our best-performing GPU (CUDA without reflexive optimization) and CPU (SIMD + OpenMP) implementations to maximize resource utilization. The `iBodies` are split: 75% are assigned to the GPU, with the remaining processed by the CPU. The GPU handles its portion using a vertically rectangular grid, with an extra tile added if the division isn't perfect, ensuring proper thread alignment. Meanwhile, the CPU begins its work where the GPU grid ends, leveraging SIMD and OpenMP for parallelism. Both computations proceed concurrently, with the CPU starting before the `cudaDeviceSynchronize()` ensures GPU completion. Results from the GPU are stored in a temporary buffer and merged with the CPU results into `accelerations`.

**Expected Performance** Compared to the standalone CUDA and SIMD + OpenMP implementations, this approach should yield better throughput, as it distributes the workload efficiently across all available resources on the Jetson TX2. However, the overall gain depends on minimizing synchronization overhead and efficiently balancing the CPU-GPU workload. In this context, the division factor can be easily adjusted to test different workload splits and optimize performance further.

Unfortunately, the use of SIMD in a `.cu` file caused compilation issues, which we suspect are related to `nvcc`. As a result, we decided to abandon this implementation. However, the code remains available in the `implem/hetero` folder for reference.

### OMP CUDA hetero

**Description** This final implementation, CUDA + OpenMP, mirrors the previous heterogeneous implementation but replaces the SIMD + OpenMP CPU component with the naive implementation enhanced with the common optimizations and multithreaded using OpenMP. This enables a version that can be implemented without relying on MIPP, while still performing better than the naive approach and retaining the same overall structure and division of work between the GPU and CPU.

We chose not to use the optimized OpenMP CPU version, despite its superior performance compared to the naive OpenMP version, because it would require pairing it with the reflexive CUDA implementation, which underperformed compared to the standard CUDA implementation. This tradeoff would have sacrificed overall performance.

**Expected Performance** This implementation is expected to deliver solid performance, comparable to the best heterogeneous approach, while slightly trailing SIMD + OpenMP due to the less optimized CPU component, but still outperforming the standalone versions.

# 3 Computer Architecture

The hardware used in our project in the Nvidia Jetson TX2, an embedded computing boards from Nvidia designed for accelerating machine learning applications. It features 1.33 TFLOPS of performance for a power of 7.5 to 15 Watt. It is based on a Tegra X2 microarchitecture and contains:

- 2 × Nvidia Denver 2 64-bit CPU
- 4 × ARM Cortex-A57
- 8 GiB of RAM
- A 256-core Nvidia Pascal architecture GPU

The Denver 2 is a 7-wide superscalar and 15-stage pipelined CPU, with a 2-way decoder. It can be considered out-of-order due to its translation of ARM code by a hardware translator.

The Cortex A57 is a 8-wide super-scalar and 18-stage pipelined CPU, with a 3-way decoder. It supports out-of-order execution with a 40-instruction window.

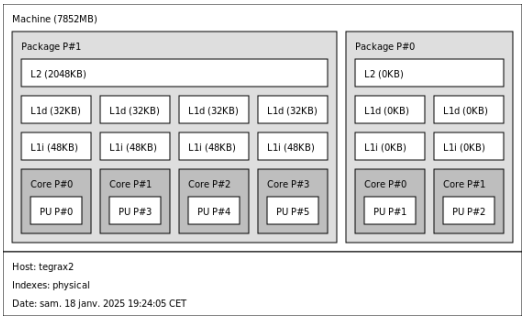



Figure 1: Nvidia Jetson TX2: Cores and Caches Topology

# 4 Experimentation Results

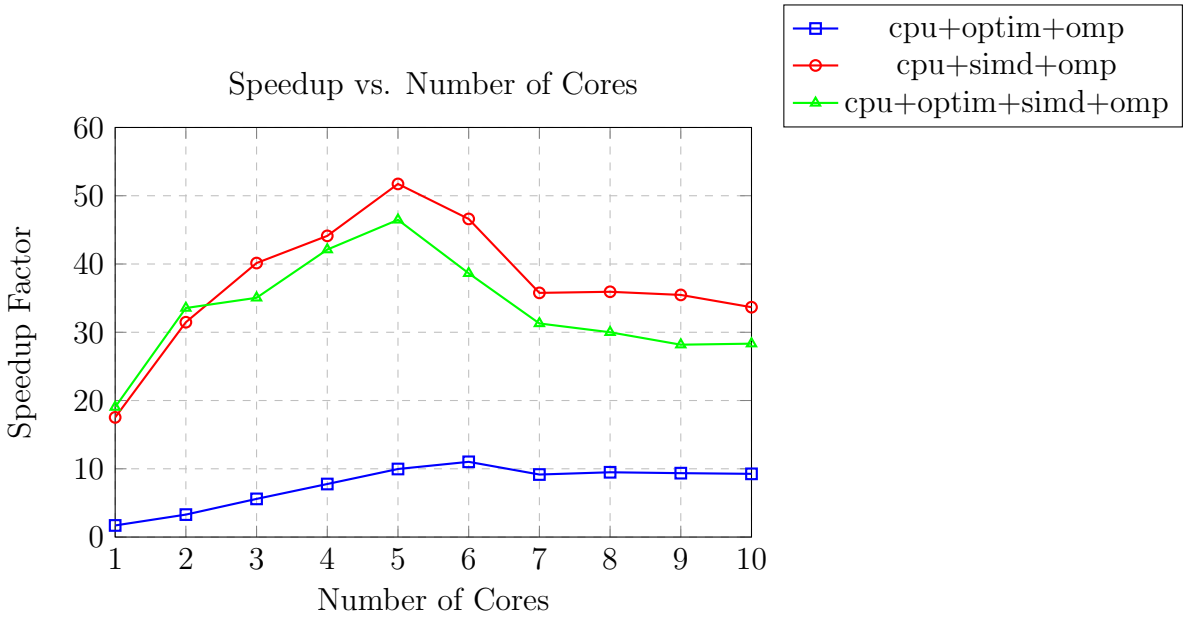
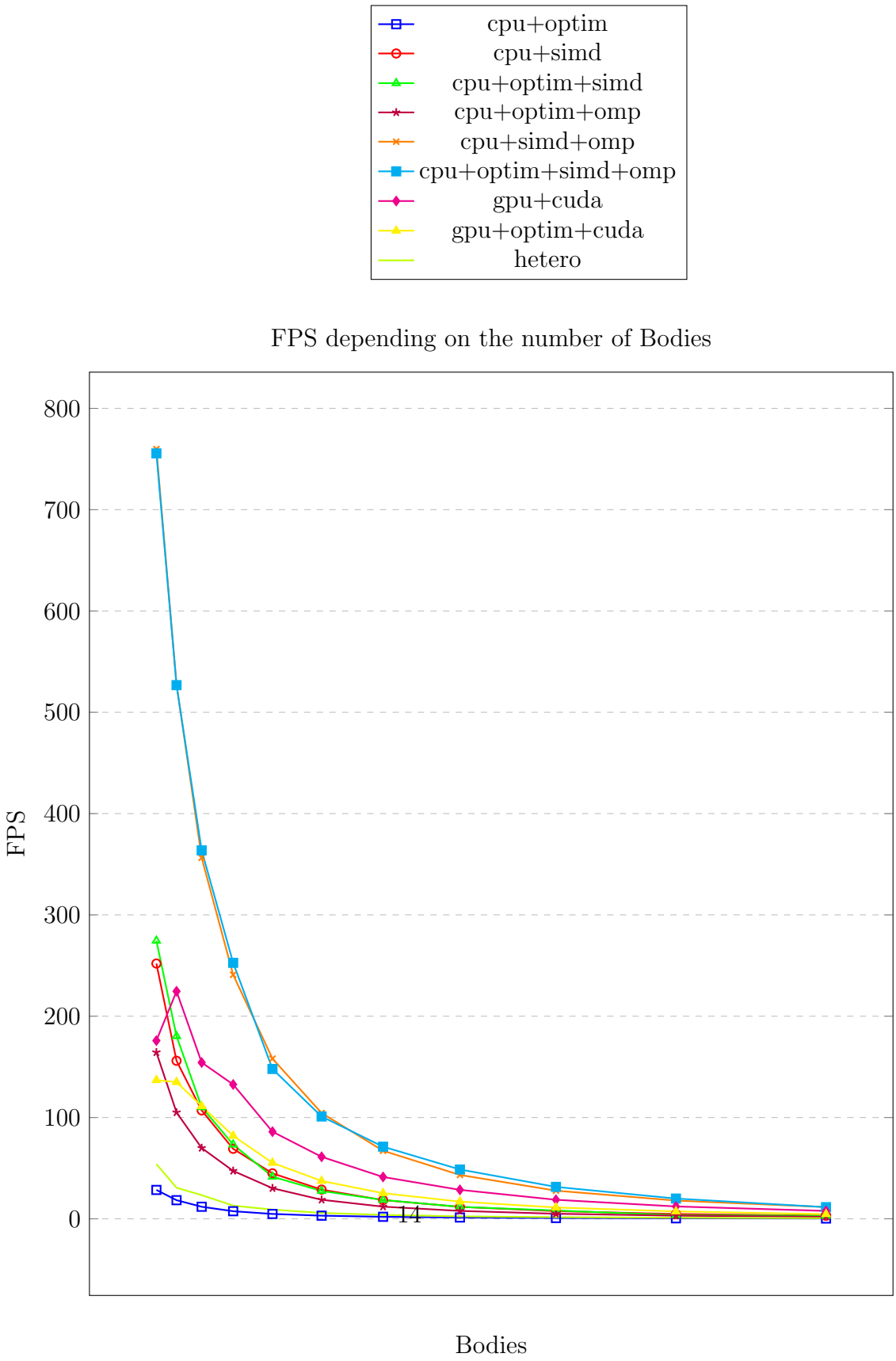
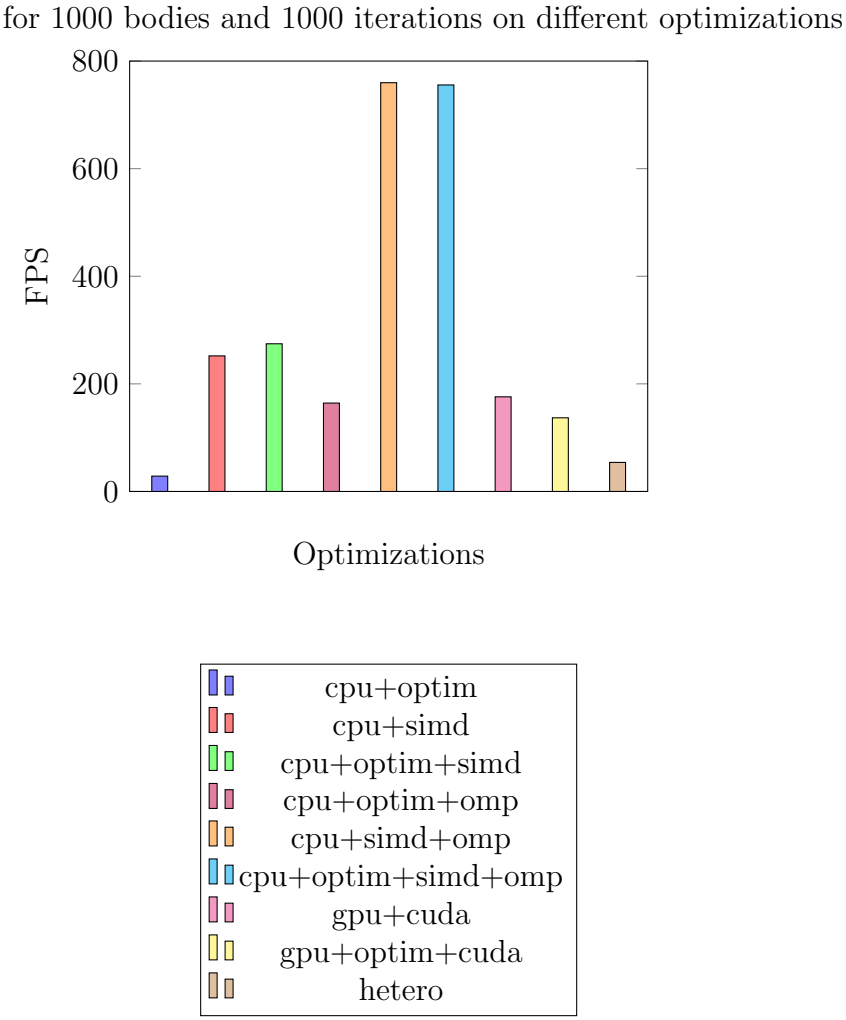



Figure 2: Speedup Factor vs. Number of Cores


cpu+optim
cpu+simd
cpu+optim+simd
cpu+optim+omp
cpu+simd+omp
cpu+optim+simd+omp
gpu+cuda
gpu+optim+cuda
hetero
FPS depending on the number of Bodies
FPS
800
700
600
500
400
300
200
100
0
Bodies

| **Index (xtick)** | **Number of bodies (increases by 25%)** |
|---|---|
| 1 | 1000 |
| 2 | 1250 |
| 3 | 1562 |
| 4 | 1953 |
| 5 | 2441 |
| 6 | 3051 |
| 7 | 3814 |
| 8 | 4768 |
| 9 | 5960 |
| 10 | 7450 |
| 11 | 9313 |

Table 1: xtick values for FPS depending on the number of bodies


FPS for 1000 bodies and 1000 iterations on different optimizations
800
600
400
200
0
FPS
Optimizations
cpu+optim
cpu+simd
cpu+optim+simd
cpu+optim+omp
cpu+simd+omp
cpu+optim+simd+omp
gpu+cuda
gpu+optim+cuda
hetero

# 5 Conclusion

Based on the results, it is evident that the SIMD + OpenMP and Optimized SIMD + OpenMP implementations are the best-performing versions. This is not entirely surprising, as the combination of multithreading and the vectorized parallelism offered by SIMD provides an edge over the GPU implementations. However, the difference between the two implementations primarily comes down to core utilization.

On a smaller number of cores, the reflexive optimization outperforms because it reduces the number of computations per thread while still handling a significant workload. On the other hand, as the number of cores increases, the division of work becomes uneven in the reflexive version, with the first threads handling significantly more work than the others. In contrast, the non-optimized version distributes the workload more evenly, keeping all threads consistently active.

Through these optimizations, we achieved a remarkable performance improvement, with the program running up to 44 times faster (from 17 fps to approximately 750 fps) using the SIMD + OpenMP implementation.

Further improvements could focus on optimizing the workload distribution among threads in the reflexive version to mitigate the imbalance. Alternatively, adopting an approximation-based algorithm could further enhance performance by significantly reducing the computational workload, though at the cost of precision, thus moving away from the strict n-body problem.